

Maven Java Web + Tomcat 10.1.x + Session + ServletContext 學習文件

學習目標

1. 建立 Maven Java Web 專案
2. 整合 Tomcat 10.1.x
3. 掌握 Session 管理機制
4. 學會使用 ServletContext
5. 了解生命週期與實際應用

1. Maven Java Web 專案結構

```
my-webapp/
├── pom.xml                    # Maven 設定檔
├── src/
│   ├── main/
│   │   ├── java/            # Java 原始碼
│   │   │   ├── com/example/
│   │   │   │   ├── servlet/
│   │   │   │   │   ├── LoginServlet.java
│   │   │   │   │   └── HomeServlet.java
│   │   │   │   ├── listener/
│   │   │   │   │   └── AppListener.java
│   │   │   │   └── model/
│   │   │   │       └── User.java
│   │   ├── resources/       # 設定檔
│   │   └── webapp/          # Web 資源
│   │       ├── WEB-INF/
│   │       │   └── web.xml   # 部署描述檔
│   │       ├── css/
│   │       ├── js/
│   │       └── index.jsp
│   └── test/
└── target/                  # 編譯輸出
```

2. POM 設定

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.example</groupId>
<artifactId>my-webapp</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>war</packaging>
<name>my-webapp</name>

<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
  <!-- Servlet API 6.0 (Tomcat 10.1.x) -->
  <dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>6.0.0</version>
    <scope>provided</scope>
  </dependency>

  <!-- JSP API 3.1 (Tomcat 10.1.x) -->
  <dependency>
    <groupId>jakarta.servlet.jsp</groupId>
    <artifactId>jakarta.servlet.jsp-api</artifactId>
    <version>3.1.0</version>
    <scope>provided</scope>
  </dependency>

  <!-- JSTL 3.0 -->
  <dependency>
    <groupId>jakarta.servlet.jsp.jstl</groupId>
    <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
    <version>3.0.0</version>
  </dependency>
  <dependency>
    <groupId>org.glassfish.web</groupId>
    <artifactId>jakarta.servlet.jsp.jstl</artifactId>
    <version>3.0.1</version>
  </dependency>
</dependencies>

<build>
  <finalName>my-webapp</finalName>
  <plugins>
    <!-- Maven War Plugin -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-war-plugin</artifactId>
      <version>3.4.0</version>
    </plugin>

    <!-- Maven Tomcat Plugin -->
```

```
<plugin>
  <groupId>org.apache.tomcat.maven</groupId>
  <artifactId>tomcat7-maven-plugin</artifactId>
  <version>2.2</version>
  <configuration>
    <path>/</path>
    <port>8080</port>
    <uriEncoding>UTF-8</uriEncoding>
  </configuration>
</plugin>
</plugins>
</build>
</project>
```

3. Tomcat 10.1.x 版本對應

Tomcat 版本	Servlet 版本	JSP 版本	EL 版本	Java 版本
10.1.x	6.0	3.1	5.0	11+
10.0.x	5.0	3.0	4.0	8+
9.0.x	4.0	2.3	3.0	8+

重要：Tomcat 10.x 開始使用 `jakarta.*` 命名空間，不再是 `javax.*`

4. web.xml 設定

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
    https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
  version="6.0">

  <display-name>我的 Web 應用</display-name>

  <!-- Session 設定 (分鐘) -->
  <session-config>
    <session-timeout>30</session-timeout>
    <cookie-config>
      <name>JSESSIONID</name>
      <http-only>true</http-only>
      <secure>false</secure>
    </cookie-config>
    <tracking-mode>COOKIE</tracking-mode>
  </session-config>

  <!-- 歡迎頁面 -->
  <welcome-file-list>
```

```
<welcome-file>index.jsp</welcome-file>
</welcome-file-list>

<!-- Servlet 設定 -->
<servlet>
    <servlet-name>LoginServlet</servlet-name>
    <servlet-class>com.example.servlet.LoginServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>LoginServlet</servlet-name>
    <url-pattern>/login</url-pattern>
</servlet-mapping>

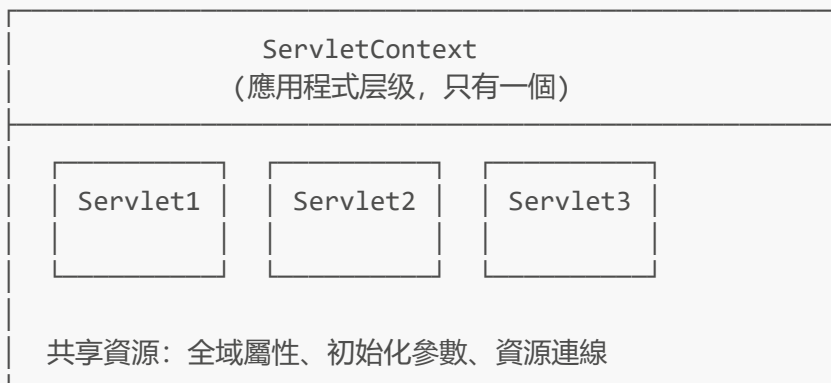
<!-- Listener 設定 -->
<listener>
    <listener-class>com.example.listener.AppListener</listener-class>
</listener>

<!-- Context 初始參數 -->
<context-param>
    <param-name>appName</param-name>
    <param-value>我的 Web 應用</param-value>
</context-param>
</web-app>
```

5. ServletContext 詳解

5.1 概念

ServletContext 代表整個 **Web 應用程式**，是所有 Servlet 共享的全域物件。



5.2 取得 ServletContext

```
// 方法一: 從 ServletConfig 取得
ServletContext context = getServletContext();
```

```
// 方法二：從 HttpServletRequest 取得
ServletContext context = request.getServletContext();
```

5.3 常用方法

```
@WebServlet("/context-demo")
public class ContextDemoServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        ServletContext context = req.getServletContext();

        // 1. 設定/取得全域屬性
        context.setAttribute("globalVar", "全域變數");
        String value = (String) context.getAttribute("globalVar");

        // 2. 取得初始化參數
        String appName = context.getInitParameter("appName");

        // 3. 取得虛擬路徑對應的真實路徑
        String realPath = context.getRealPath("/WEB-INF");

        // 4. 取得 MIME 類型
        String mimeType = context.getMimeType("file.pdf");

        // 5. 取得版本資訊
        int majorVersion = context.getMajorVersion();
        int minorVersion = context.getMinorVersion();

        // 6. 記錄日誌
        context.log("這是一則日誌訊息");

        // 輸出結果
        res.setContentType("text/html;charset=UTF-8");
        PrintWriter out = res.getWriter();
        out.println("<h2>ServletContext Demo</h2>");
        out.println("<p>應用程式名稱: " + appName + "</p>");
        out.println("<p>全域變數: " + value + "</p>");
        out.println("<p>真實路徑: " + realPath + "</p>");
        out.println("<p>MIME 類型: " + mimeType + "</p>");
        out.println("<p>Servlet 版本: " + majorVersion + "." + minorVersion + "
</p>");
    }
}
```

5.4 實際應用範例：網站訪客統計

```
package com.example.listener;

import jakarta.servlet.ServletContext;
import jakarta.servlet.ServletContextEvent;
import jakarta.servlet.ServletContextListener;
import jakarta.servlet.annotation.WebListener;

@WebListener
public class VisitorListener implements ServletContextListener {

    @Override
    public void contextInitialized(ServletContextEvent sce) {
        ServletContext context = sce.getServletContext();

        // 初始化計數器
        context.setAttribute("totalVisitors", 0);
        context.setAttribute("onlineUsers", 0);

        // 初始化應用程式啟動時間
        context.setAttribute("startTime", System.currentTimeMillis());

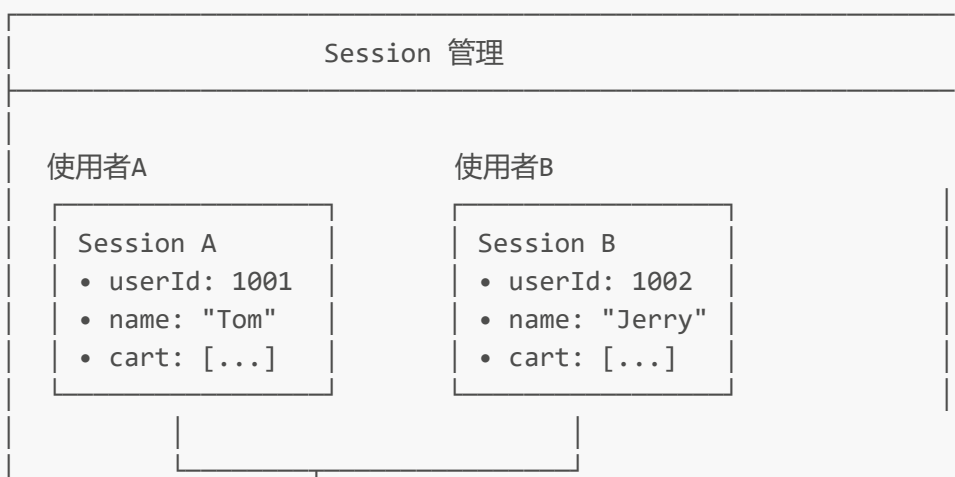
        context.log("VisitorListener 初始化完成");
    }

    @Override
    public void contextDestroyed(ServletContextEvent sce) {
        sce.getServletContext().log("VisitorListener 關閉");
    }
}
```

6. Session 詳解

6.1 概念

HttpSession 代表單一使用者的會話狀態，用於儲存使用者相關資料。



▼
Server (儲存在記憶體)

6.2 Session 生命週期

使用者第一次造訪



```
HttpSession session =
    request.getSession(true);
```

← 建立新 Session



Session 使用中...
(可儲存/取得屬性)



```
session.getAttribute()
session.setAttribute()
```



Session 超時 (預設30分鐘)
或 手動 invalidate()



```
session.invalidate();
```

← 銷毀 Session



Session 銷毀, 資料清除

6.3 常用方法

```
@WebServlet("/session-demo")
public class SessionDemoServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        // 1. 取得 Session (不存在則建立)
        HttpSession session = req.getSession(true);

        // 2. 取得 Session ID
        String sessionId = session.getId();
```

```

// 3. 設定屬性
session.setAttribute("username", "Tom");
session.setAttribute("loginTime", new java.util.Date());

// 4. 取得屬性
String username = (String) session.getAttribute("username");

// 5. 移除屬性
session.removeAttribute("loginTime");

// 6. 取得/設定超時時間 (秒)
int maxInactive = session.getMaxInactiveInterval();
session.setMaxInactiveInterval(60 * 30); // 30 分鐘

// 7. 判斷是否為新 Session
boolean isNew = session.isNew();

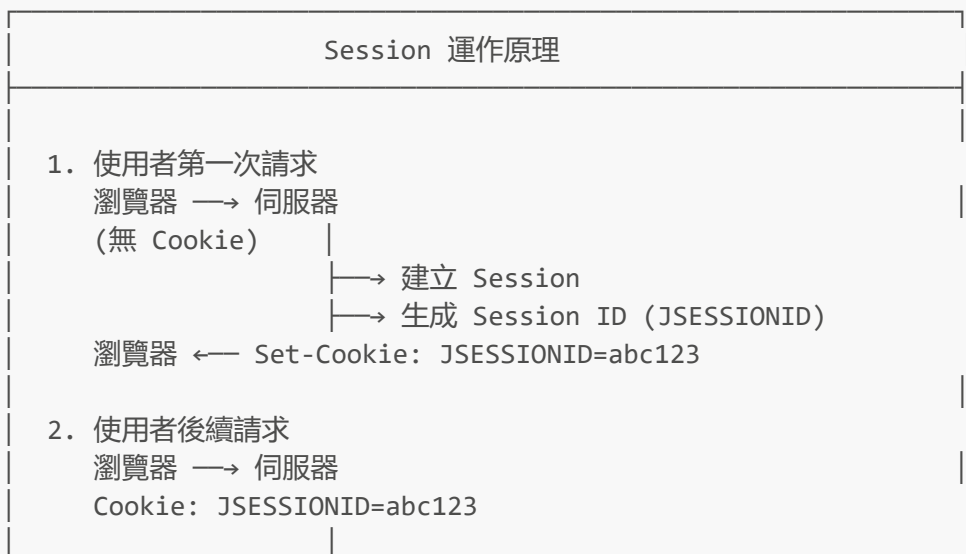
// 8. 取得建立時間
long createTime = session.getCreationTime();

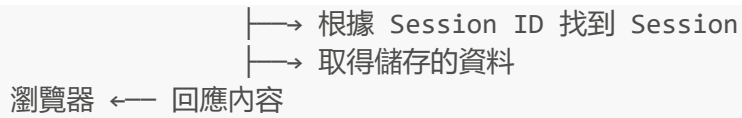
// 9. 取得最後存取時間
long lastAccess = session.getLastAccessedTime();

// 輸出結果
res.setContentType("text/html;charset=UTF-8");
PrintWriter out = res.getWriter();
out.println("<h2>Session Demo</h2>");
out.println("<p>Session ID: " + sessionId + "</p>");
out.println("<p>使用者名稱: " + username + "</p>");
out.println("<p>是否新 Session: " + isNew + "</p>");
out.println("<p>超時時間: " + maxInactive + " 秒</p>");
}
}

```

6.4 Session 與 Cookie 的關係





7. 完整範例：登入系統

7.1 User.java

```

package com.example.model;

import java.io.Serializable;

public class User implements Serializable {
    private int id;
    private String username;
    private String name;
    private String role;

    public User() {}

    public User(int id, String username, String name, String role) {
        this.id = id;
        this.username = username;
        this.name = name;
        this.role = role;
    }

    // Getter & Setter
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getRole() { return role; }
    public void setRole(String role) { this.role = role; }
}

```

7.2 LoginServlet.java

```

package com.example.servlet;

import com.example.model.User;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

```

```
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;

@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    // 模擬資料庫
    private static final Map<String, User> users = new HashMap<>();

    static {
        users.put("admin", new User(1, "admin", "管理員", "admin"));
        users.put("tom", new User(2, "tom", "Tom", "user"));
        users.put("jerry", new User(3, "jerry", "Jerry", "user"));
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        // 檢查是否已登入
        HttpSession session = req.getSession(false);
        if (session != null && session.getAttribute("user") != null) {
            res.sendRedirect(req.getContextPath() + "/home");
            return;
        }

        // 顯示登入頁面
        req.getRequestDispatcher("/login.jsp").forward(req, res);
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        String username = req.getParameter("username");
        String password = req.getParameter("password");

        // 驗證帳號密碼
        User user = users.get(username);
        if (user != null && "1234".equals(password)) {
            // 登入成功, 建立 Session
            HttpSession session = req.getSession(true);
            session.setAttribute("user", user);
            session.setAttribute("loginTime", new java.util.Date());

            // 設定 Session 超時時間 (30分鐘)
            session.setMaxInactiveInterval(30 * 60);

            // 導向首頁
            res.sendRedirect(req.getContextPath() + "/home");
        } else {
            // 登入失敗
            req.setAttribute("error", "帳號或密碼錯誤");
        }
    }
}
```

```
        req.getRequestDispatcher("/login.jsp").forward(req, res);
    }
}
```

7.3 HomeServlet.java

```
package com.example.servlet;

import com.example.model.User;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import java.io.IOException;

@WebServlet("/home")
public class HomeServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        HttpSession session = req.getSession(false);

        // 檢查是否已登入
        if (session == null || session.getAttribute("user") == null) {
            res.sendRedirect(req.getContextPath() + "/login");
            return;
        }

        // 取得使用者資訊
        User user = (User) session.getAttribute("user");
        req.setAttribute("user", user);

        // 顯示首頁
        req.getRequestDispatcher("/home.jsp").forward(req, res);
    }
}
```

7.4 LogoutServlet.java

```
package com.example.servlet;

import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import java.io.IOException;

@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {
```

```
@Override
protected void doGet(HttpServletRequest req, HttpServletResponse res)
    throws IOException {

    HttpSession session = req.getSession(false);
    if (session != null) {
        session.invalidate(); // 銷毀 Session
    }

    res.sendRedirect(req.getContextPath() + "/login");
}
}
```

7.5 SessionListener.java

```
package com.example.listener;

import jakarta.servlet.annotation.WebListener;
import jakarta.servlet.http.*;

@WebListener
public class SessionListener implements HttpSessionListener {

    @Override
    public void sessionCreated(HttpSessionEvent se) {
        HttpSession session = se.getSession();
        System.out.println("Session 建立: " + session.getId());

        // 更新線上人數
        var context = session.getServletContext();
        Integer onlineUsers = (Integer) context.getAttribute("onlineUsers");
        onlineUsers = (onlineUsers == null) ? 0 : onlineUsers;
        context.setAttribute("onlineUsers", onlineUsers + 1);
    }

    @Override
    public void sessionDestroyed(HttpSessionEvent se) {
        HttpSession session = se.getSession();
        System.out.println("Session 銷毀: " + session.getId());

        // 更新線上人數
        var context = session.getServletContext();
        Integer onlineUsers = (Integer) context.getAttribute("onlineUsers");
        onlineUsers = (onlineUsers == null) ? 0 : onlineUsers;
        context.setAttribute("onlineUsers", Math.max(0, onlineUsers - 1));
    }
}
```

7.6 login.jsp

```

<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<html>
<head>
  <title>登入</title>
  <style>
    body { font-family: Arial; max-width: 400px; margin: 100px auto; }
    .error { color: red; margin-bottom: 10px; }
    .form-group { margin-bottom: 15px; }
    label { display: block; margin-bottom: 5px; }
    input { width: 100%; padding: 8px; box-sizing: border-box; }
    button { background: #007bff; color: white; padding: 10px 20px; border:
none; cursor: pointer; }
  </style>
</head>
<body>
  <h2>會員登入</h2>

  <% if (request.getAttribute("error") != null) { %>
    <div class="error">${error}</div>
  <% } %>

  <form action="${pageContext.request.contextPath}/login" method="post">
    <div class="form-group">
      <label>帳號: </label>
      <input type="text" name="username" required>
    </div>
    <div class="form-group">
      <label>密碼: </label>
      <input type="password" name="password" required>
    </div>
    <button type="submit">登入</button>
  </form>

  <p>測試帳號: admin / tom / jerry<br>密碼: 1234</p>
</body>
</html>

```

7.7 home.jsp

```

<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<html>
<head>
  <title>首頁</title>
  <style>
    body { font-family: Arial; max-width: 600px; margin: 50px auto; }
    .info { background: #f5f5f5; padding: 20px; border-radius: 5px; }
    .logout { background: #dc3545; color: white; padding: 8px 15px; text-
decoration: none; }
  </style>

```

```
</head>
<body>
  <h2>歡迎, ${user.name}! </h2>

  <div class="info">
    <p><strong>帳號: </strong>${user.username}</p>
    <p><strong>姓名: </strong>${user.name}</p>
    <p><strong>角色: </strong>${user.role}</p>
    <p><strong>登入時間: </strong>${sessionScope.loginTime}</p>
    <p><strong>Session ID: </strong>${session.id}</p>
    <p><strong>目前線上人數: </strong>${applicationScope.onlineUsers}</p>
  </div>

  <br>
  <a href="${pageContext.request.contextPath}/logout" class="logout">登出</a>
</body>
</html>
```

8. ServletContext vs Session 比較

特性	ServletContext	HttpSession
範圍	全應用程式 (所有使用者共享)	單一使用者 (個別獨立)
生命週期	應用程式啟動 ~ 關閉	使用者登入 ~ 登出/超時
儲存位置	伺服器記憶體	伺服器記憶體
主要用途	全域設定、統計資料	使用者資料、購物車
取得方式	<code>request.getServletContext()</code>	<code>request.getSession()</code>
屬性數量	1 個 ServletContext	N 個 Session (N=使用者數)

圖解比較



9. Session 常見問題

9.1 Session 超時設定

```
<!-- web.xml -->
<session-config>
    <session-timeout>30</session-timeout>  <!-- 30 分鐘 -->
</session-config>
```

```
// Java 程式碼
session.setMaxInactiveInterval(30 * 60); // 30 分鐘 (單位: 秒)
```

9.2 Session 銷毀時機

情況	說明
超時	超過 <code>maxInactiveInterval</code> 時間未使用
手動銷毀	呼叫 <code>session.invalidate()</code>
伺服器關閉	伺服器停止時 (除非設定持久化)

9.3 Session 持久化

```
// 在伺服器重啟後保留 Session
// Tomcat 設定: server.xml
<Manager className="org.apache.catalina.session.PersistentManager">
    <Store className="org.apache.catalina.session.FileStore"/>
</Manager>
```

10. 執行與測試

10.1 Maven Tomcat Plugin 執行

```
# 執行 Tomcat
mvn tomcat7:run
```

```
# 或打包後部署
mvn clean package
```

10.2 部署到 Tomcat

```
# 複製 war 檔案到 Tomcat
cp target/my-webapp.war $CATALINA_HOME/webapps/

# 啟動 Tomcat
$CATALINA_HOME/bin/startup.sh # Linux/Mac
$CATALINA_HOME/bin/startup.bat # Windows
```

10.3 測試網址

```
http://localhost:8080/my-webapp/      # 首頁
http://localhost:8080/my-webapp/login # 登入頁面
http://localhost:8080/my-webapp/home  # 首頁 (需登入)
http://localhost:8080/my-webapp/logout # 登出
http://localhost:8080/my-webapp/context-demo # ServletContext 測試
http://localhost:8080/my-webapp/session-demo # Session 測試
```

11. 常見錯誤

錯誤	原因	解決方法
<code>ClassNotFoundException</code>	缺少 Servlet API	確認 pom.xml 有 <code>jakarta.servlet-api</code>
<code>404 Not Found</code>	URL 對應錯誤	檢查 <code>@WebServlet</code> 或 <code>web.xml</code>
Session 資料遺失	Session 超時或瀏覽器關閉 Cookie	延長超時時間或使用 URL 重寫
<code>IllegalStateException</code>	Session 已被銷毀	先檢查 Session 是否存在

12. 學習路徑

```
1. Maven 基礎
  |
  ▼
2. Servlet 6.0 基礎
  |
  ▼
3. Tomcat 10.1.x 部署
  |
```



參考資源

- Tomcat 10.1 官方文件: <https://tomcat.apache.org/tomcat-10.1-doc/>
- Jakarta Servlet 6.0: <https://jakarta.ee/specifications/servlet/>
- Maven Tomcat Plugin: <https://tomcat.apache.org/tomcat-7.0-doc/maven-howto.html>